



Orientacao a Objetos em Dart

Desenvolvimento Móvel

Aula 04/36

Flutter

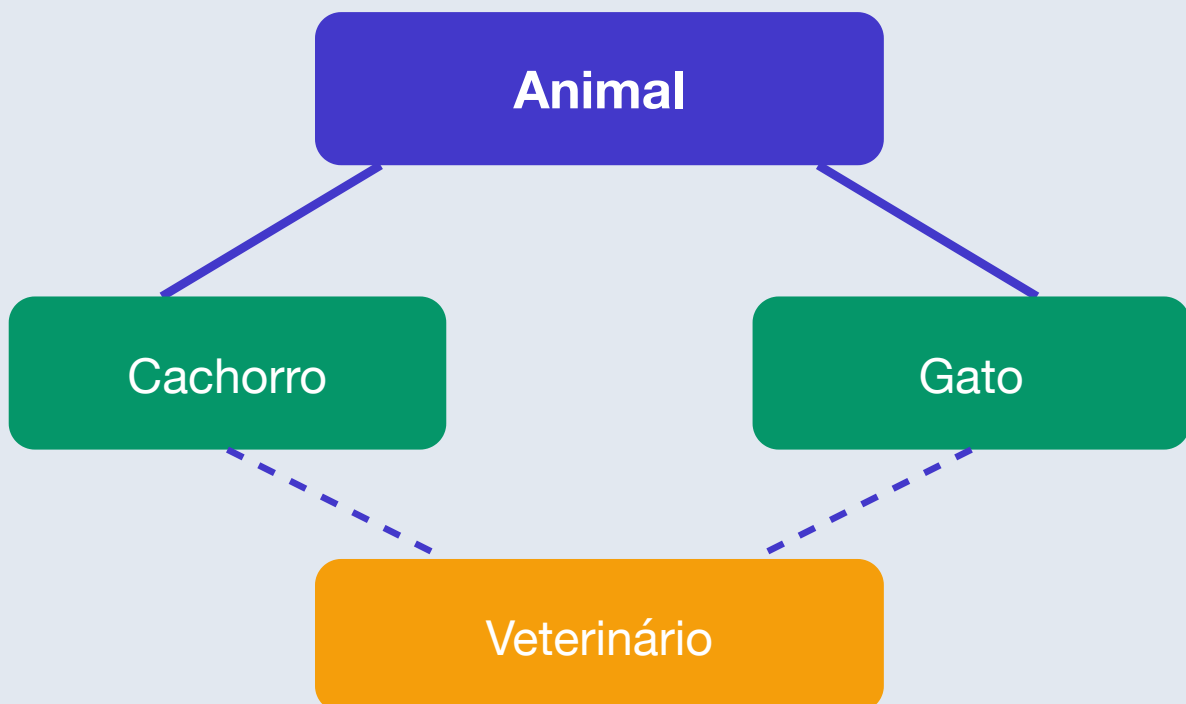
Dart

Widget

Classe

Conceito

Dart trata tudo como objeto — até números como `int` e `bool` herdam de `Object`, como se toda peça de LEGO se encaixasse no mesmo sistema. Classes são como moldes com campos, construtores, getters/setters e métodos. O construtor com sintaxe de inicialização (`class Pessoa { String nome; Pessoa(this.nome); }`) economiza linhas de código. Herança única com `extends` permite um filho aproveitar features do pai, com `@override` para personalizar métodos. Mixins (`with`) são como "capsulas de habilidades" que você acopla a uma classe sem precisar de herança múltipla, muito usados no Flutter (`WidgetsBindingObserver`, `AutomaticKeepAliveClientMixin`). Interfaces usam `implements`, podendo uma classe assinar múltiplos contratos. Generics (`List<int>`, `Map<String, dynamic>`) garantem type safety em coleções, como etiquetas que dizem exatamente o que cabe em cada caixa.





Atividade Prática

Crie uma hierarquia de classes: uma classe abstrata `Animal` com método `void emitirSom()` e um atributo `String nome`. Implemente `Cachorro` (que late) e `Gato` (que mia). Crie uma classe `Veterinario` com método `void examinar(Animal a)` que chama `emitirSom()` do animal. No `main()`, crie uma lista `List<Animal>` com diferentes animais e use polimorfismo para chamar `examinar()` em cada um. Adicione um mixin chamado `Domesticavel` com método `void levarParaPassear()`.



Pesquisa

Pesquise sobre mixins em Dart além do básico: restrições (on keyword), mixins que estendem classes, diferenças entre mixin e mixin class (Dart 3). Como os mixins resolvem o problema do diamante em herança múltipla? Exemplos de uso real no Flutter SDK.



Requisitos

Dart SDK. Testar com: `dart run arquivo.dart`.